

实验一：AES 分组密码的 ECB 模式安全性分析

—— 一次不需要密钥的密文伪造

项目	内容
实验名称	AES 分组密码的 ECB 模式安全性分析
实验类型	程序设计类（攻击复现 + 防御对照）
前置知识	分组密码基本概念、异或运算、Python 基础
编程环境	Python 3.8+, cryptography 库

0. 实验速览

一句话概括

你会在**完全不知道密钥**的情况下，把 Alice 发给 Bob 的一条加密消息，改成一条 Alice 从来没发过的消息，而且 Bob 收到后看不出任何异常。

这为什么可能

不是 AES 被破解了 —— AES 本身完好无损。问题出在**工作模式**上。

ECB (Electronic Codebook, 电码本模式) 把每条消息切成 16 字节的块，每块**独立地**用 AES 加密。于是密文块就像一盒乐高积木：不带密钥也能拆下来，换到别的地方去。

你会做什么

任务	内容	性质
任务一	看清密文的分组结构	观察
任务二	发现 ECB 的模式泄露	观察
任务三	第一次搬运分组，理解“为什么一开始会失败”	核心
任务四	写一个交互式密文伪造工具	主交付物
任务五	挑战：拼出一条从未存在过的消息	挑战
任务六	为什么换成 CBC / GCM 就防住了	防御

交付物

- 任务四的伪造程序（可在任务四基础上完善）
- 一份实验报告（格式见 第 5 节）

 **第一次上手？** 先看 [实验步骤.md](#) —— 那份文件按“先敲什么、该看到什么”的顺序写的，可以直接照着做。本文件负责讲清楚**为什么**。

1. 原理

1.1 分组密码：把 16 字节变成 16 字节

AES 是一个**分组密码** (block cipher)。它的接口非常单纯：

```
AES_加密(密钥, 16字节明文) -> 16字节密文
AES_解密(密钥, 16字节密文) -> 16字节明文
```

就这么多。它一次只吃 16 字节，也只吐 16 字节。

在**固定密钥**下，AES 的加密函数是 $\mathbb{F}_2^{128} \rightarrow \mathbb{F}_2^{128}$ 上的一个**置换**（——映射）。这个置换由密钥唯一确定，且没有密钥就无从计算其逆——这就是分组密码安全性的全部内容。

注意这里有个关键词：**固定密钥**。下面的推论是本实验的理论基础：

推论（ECB 的确定性） 密钥固定时，AES 是一个确定的函数。因此：**相同的明文分组，必然产生相同的密文分组。**

这条推论看起来平淡无奇，但它是后面所有攻击的起点。

1.2 工作模式：分组密码不够用

现实中的消息远比 16 字节长。要把分组密码用在长消息上，必须规定：

- 怎么切块
- 每块怎么加密
- 块与块之间有没有关系

这套规定就叫**工作模式**（mode of operation）。常见的几种：

模式	核心思想	有完整性保护吗
ECB	每块独立加密	✗
CBC	每块的输入先与前一块的密文异或	✗
CTR	把分组密码当流密码用	✗
GCM	CTR + 认证标签	✓
ChaCha20-Poly1305	流密码 + 认证标签	✓

关键认识：ECB、CBC、CTR 只解决“怎么加密”，**不解决“怎么防止被改”**。只有最后两类（统称 AEAD，带关联数据的认证加密）才同时提供机密性和完整性。

1.3 ECB 的定义

ECB 是所有模式里最简单的：

密文块[i] = AES_加密(密钥, 明文块[i])

就这一行。没有初始化向量（IV），没有链式关系，没有计数器。每一块都是**完全独立**的一次 AES 调用。

工程上它还有个诱人的好处：**可以并行计算**，加密快、实现简单。这正是它至今仍在一些老旧系统里出现的原因。

1.4 ECB 的两个致命性质

性质一：确定性 —— 相同明文块 → 相同密文块

由 1.1 的推论直接得到。后果是**模式泄露**：

攻击者不需要密钥，只要横向比对多条密文的分组，就能判断出「这两条消息是同一个人发的」「这两条是同一天」，甚至「这两条消息除了最后一个字段，其余完全一样」。

也就是说，加密**没有隐藏消息的结构**。

性质二：无完整性 —— 密文块可以自由搬运

因为块与块之间没有任何绑定，攻击者可以：

- 调换密文块的位置
- 删除密文块
- 把一个密文的块搬到另一个密文里

而**解密方无法察觉**：解密程序只是忠实地把每个块解密，然后拼起来。

这就是本实验要复现的攻击，学名是**密文分组拼接攻击**（ciphertext block splicing / cut-and-paste attack）。

1.5 为什么这个攻击值得认真对待

初学者常有一个误解：“**加密算法安全 = 加密是安全的**”。

本实验要打破的正是这个误解。看看攻击面：

攻击者知道什么	攻击者不知道什么
密文（当然）	密钥
消息的大致格式与字段含义	本条的完整内容

历史上传输过的其他密文

结论是：在不知道密钥的情况下，攻击者已经能够伪造出语义合理的新消息。

而且这个攻击**极其廉价**：不需要算力，不需要数学，只是搬 16 字节的内存块。一个能读取网络流量的人，几行代码就能做到。

1.6 本实验的攻击模型

角色	拥有什么	做不到什么
Alice（发送方）	密钥	—
Bob（接收方）	密钥	无法判断密文是否被改过
Mallory（攻击者）	只能看到密文，知道消息格式和历史记录	没有密钥，无法解密

这是个标准的**已知明文场景**：Mallory 知道（或猜到）历史消息的内容，因为这本来就是低熵的字段——人名、日期、地点，猜起来不难。

⚠️ 请注意区分两件事：**Mallory 不需要知道明文才能搬运分组**——他只需要知道“哪一块对应哪个字段”。知道明文内容只是帮他**挑得更准**，不是攻击成立的前提。

2. 实验环境

2.1 需要什么

- Python 3.8 或更高
- cryptography 库

安装：

```
pip install cryptography
```

💡 如果系统里同时有多个 Python，用 `python -m pip install cryptography` 更稳妥，能确保装到你实际运行的那个解释器里。

操作系统：Windows / macOS / Linux 都可以，**不需要 Linux**，也不需要 WSL。全部代码只用 Python 标准库和 cryptography，没有任何平台相关的调用，路径也全部用 `pathlib` 处理。若在 Windows 的 `cmd.exe` 里遇到中文显示问题，见 [A.4](#)。

2.2 目录结构

```
aes-lab/
├── 实验指导.md    <- 本文件
├── 实验原理.md    <- 实验所需理论知识
├── 实验步骤.md    <- 照着敲的版本（讲“怎么做”）
├── code/
│   ├── run_all.py        一键跑通全部任务（便于批改）
│   ├── check_env.py      环境自检（用官方测试向量验证）
│   ├── block_utils.py    只做字节搬运，无密钥、无解密
│   ├── ecb_common.py     密钥 / 消息格式 / ECB·CBC·GCM 封装
│   ├── alice_encrypt.py  Alice 的加密程序
│   ├── bob_decrypt.py    Bob 的解密程序
│   ├── task1_basics.py   任务一：分组结构
│   ├── task2_pattern.py  任务二：模式泄露
│   ├── task3_splICE.py   任务三：分组搬运
│   ├── task4_forge.py    任务四：交互式伪造工具【主交付物】
│   ├── task5_challenge.py 任务五：挑战题
│   └── task6_defense.py  任务六：防御对照
├── data/           运行后生成（密文、历史密文库）
└── tools/          真值生成与验证工具（学生不必使用，见 2.4）
```

`block_utils.py` 与 `ecb_common.py` 的分离是本实验的设计要点（见 [3.3](#)）。

2.3 环境自检

先做这一步。它用 FIPS-197 的官方测试向量确认你的环境算得对：

```
cd code
python check_env.py
```

期望输出：

FIPS-197 附录 C.1 已知答案测试

密钥	: 000102030405060708090a0b0c0d0e0f	
明文	: 00112233445566778899aabbccddeeff	
期望密文	: 69c4e0d86a7b0430d8cdb78070b4c55a	
实际密文	: 69c4e0d86a7b0430d8cdb78070b4c55a	通过
解密回明文	: 00112233445566778899aabbccddeeff	通过

PKCS#7 填充 : 通过

=====
环境自检全部通过，可以开始实验。

只要这里输出一致，后面所有结果就都可信。

然后建立历史密文库：

```
python alice_encrypt.py --build-history
```

2.4 关于 tools/ 目录

tools/ 里有本实验包的**工具**，学生**不需要使用**，仅作参考：

文件	用途
aes_ref.py	纯标准库 AES/CBC 实现，通过 FIPS-197 与 NIST SP 800-38A 官方向量
gen_truth.py	生成 附录 B 的真值数据
_devshim/	cryptography 的接口垫片，让本实验包在 没装 cryptography 的机器上也能端到端跑通

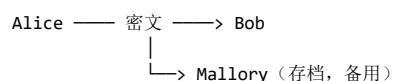
它们的价值在于：**附录 B 里的每一个十六进制数都是真跑出来的**，而且可以在任何机器上离线复核。AES-ECB 是确定性算法，所以你自己跑出来的密文与附录 B **逐字节相同** —— 如果不同，那是环境有问题，不是数据有问题。

3. 实验场景

3.1 通信模型

Alice 和 Bob 是同事，用一套**加密便签**程序互相发送约见消息。程序用 AES-ECB 加密，双方共享一个密钥。

Mallory 是网络管理员，能看到所有传输的密文，并把**历史密文都存了下来**。



3.2 消息格式

实验中会用到两种格式，它们的对比正是任务三的核心。

格式 A：自然写法

```
f"From: {sender}\nTo: {to}\nDate: {date}\nPlace: {place}"
```

例：

```
From: Alice
To: Bob
Date: 2026-03-15
Place: Library
```

字段长度**不定**。这样写最自然，但分组边界与字段边界**不对齐**。

格式 B：定长对齐

让每个字段**恰好占一个 16 字节分组**：

字段	前缀	内容宽度	该行总长
From	"From: " (6)	10	16
To	"To: " (4)	12	16
Date	"Date: " (6)	10	16
Place	"Place: " (7)	8	15

明文总长 63 字节，PKCS#7 补 1 字节 0x01 凑满 64 字节 = 4 个分组。

分组 0: "From: Alice " <- 发件人
分组 1: "To: Bob " <- 收件人
分组 2: "Date: 2026-03-15" <- 日期
分组 3: "Place: Library " + 0x01 <- 地点 (+ 填充)

一个字段 = 一个分组。这是攻击能干净成立的前提。

🔥 停一下想一想：格式 B 显然比格式 A「更好」—— 解析简单、不用处理变长字段、不容易写出 bug。但正是这个“好”，把消息变成了一盒边界清晰的乐高积木。这个矛盾是本实验最值得回味的地方。

3.3 密钥放在哪

这是本实验在工程组织上刻意做的一件事：

文件	有密钥吗	能加解密吗
ecb_common.py	✅ 有	✅ 能
alice_encrypt.py	✅ 用	✅ 能
bob_decrypt.py	✅ 用	✅ 能
block_utils.py	❌ 没有	❌ 不能
task4_forge.py	❌ 没有	❌ 不能

task4_forge.py (Mallory 的程序) 只 import block_utils，不 import ecb_common，因此在代码层面就拿不到密钥。

这不是障眼法，而是让“不需要密钥也能伪造”这句话变成可检查的事实：

```
python -c "import ast; t=ast.parse(open('task4_forge.py',encoding='utf-8').read()); \
print(sorted({n.module for n in ast.walk(t) if isinstance(n,ast.ImportFrom)}))"
```

实测输出：

```
['__future__', 'block_utils', 'pathlib']
```

作为对照，对有密钥的 bob_decrypt.py 跑同一条命令：

```
['__future__', 'ecb_common', 'pathlib']
```

区别就在第三个名字：攻击者程序用的是 block_utils（纯字节搬运），而接收方程序用的是 ecb_common（含密钥与加解密）。

4. 实验任务

任务一：看清密文的分组结构

目的：建立“密文 = 16 字节一组的方块”这个直觉。后面每一步都建立在这里。

```
python task1_basics.py
```

要求：跑通程序，然后回答：

- 明文 51 字节，密文为什么是 64 字节？多出来的 13 字节是什么？
- 如果明文长度恰好是 16 的整数倍，密文会是多少字节？（动手改代码验证）
- 把 cipher[16:32] + cipher[0:16] + cipher[32:] 解密，为什么没有报错？解出来的明文是什么样子的？

关键结论：单看一个分组，AES 是足够强的（改 1 位密钥，整条密文全变）。问题不在 AES，在**分组之间彼此独立**这件事上。

任务二：ECB 的模式泄露

目的：体会“确定性”带来的信息泄露。这一关**不用任何加解密函数**，纯粹是拿眼睛比对密文。

```
python task2_pattern.py
```

程序会给出 6 条历史密文的分组比对结果。实测输出：

分组	不同的密文分组共几种	重复的分组出现在
分组0 (From)	3 种	08096a96...b4b1a 出现在 M1, M2, M3, M6
分组1 (To)	4 种	43f16df7...2f60b 出现在 M1, M2, M3
分组2 (Date)	3 种	90cd57ec...e40681 出现在 M1, M2, M4, M6
分组3 (Place)	4 种	050d523d...95f9ea 出现在 M1, M3, M6

要求：

- 1. Mallory 仅凭这些密文，能推断出哪些事实？请逐条列出。
- 2. M1 和 M2 的密文前三个分组完全相同 —— 这说明这两条消息有什么关系？
- 3. **思考：**如果把加密换成 CBC（每次用随机 IV），上面哪些推断会失效？哪些仍然成立？

关键结论：ECB 是确定性的，确定性就是可比较性，可比较性就是信息泄露。密钥仍然是安全的，但**消息的结构**已经暴露。

任务三：第一次搬运分组

目的：理解“分组边界必须与字段边界对齐”这个关键条件。这是全实验最关键的一关。

```
python task3_ssplice.py
```

3.1 在格式 A 下尝试

程序先按格式 A 加密两条消息：

- M1: Alice -> Bob, 2026-03-15, Library (51 字节, 4 个分组)
- M2: Alice -> Bob, 2026-03-15, Gym (47 字节, **只有 3 个分组**)

把 M2 的最后一个分组接到 M1 上，实测解密结果是：

```
From: Alice
To: Bob
Date: 2026-03-15
Place: Libr3-15
Place: Gym
```

这显然不对劲。地点变成了 Libr3-15，还凭空多出一行。

为什么？把 M1 的明文按 16 字节切开就清楚了：

```
分组0: |From: Alice·To: |
分组1: |Bob·Date: 2026-0|
分组2: |3-15·Place: Libr|
分组3: |ary           | （后面是 13 字节填充）
```

分组边界**切在了字段的中间**。「From: Alice」后面的换行和「To:」被切到了下一组；地点的前半截在第 2 组、后半截在第 3 组。

所以任何一次搬运，都会**同时动到相邻的两个字段** —— 除非碰巧两条消息在切点两侧完全一样。

请动手验证：把 M3（2026-03-22）的第 2 分组搬进 M1，日期会变成 03-22。但这一次“成功”是运气：分组 2 里同时装着日期的尾巴和地点的开头，能对上只因为两条消息其余部分相同。

3.2 改成格式 B，再试一次

程序接着改用格式 B 加密：

```
M1 密文（格式 B）
分组0: 08096a96a3a0475152d184e5931b4b1a
分组1: 43f16df7956aa0ba8a53ece25932f60b
分组2: 90cd57ec0e9013c73633feca93e40681
分组3: 050d523d65fc1ba9c33a5f9cc695f9ea    <- Place: Library

M2 密文（格式 B）
分组0: 08096a96a3a0475152d184e5931b4b1a    <- 与 M1 相同
分组1: 43f16df7956aa0ba8a53ece25932f60b    <- 与 M1 相同
分组2: 90cd57ec0e9013c73633feca93e40681    <- 与 M1 相同
分组3: afed40e112212decde60a35b5fc8422e    <- Place: Gym（不同）
```

把 M2 的分组 3 搬进 M1:

```
伪造密文 = 08096a96...b4b1a 43f16df7...2f60b 90cd57ec...e40681 afed40e1...8422e
```

解密结果:

```
From: Alice
To: Bob
Date: 2026-03-15
Place: Gym
```

干净利落。地点从 Library 变成了 Gym，消息完全通顺，而 Mallory 从头到尾没有密钥。

要求:

1. 解释: 为什么格式 A 下搬运会“切坏”字段，而格式 B 下不会?
2. 格式 A 下把 M1 的**最后一个**分组换成 M3 的最后一个分组，明文会变吗? 为什么? (提示: 想想两个分组的实际内容)
3. **思考:** 如果你是这个系统的设计师，在不知道本实验结论的前提下，你会选格式 A 还是格式 B? 为什么? 这个选择说明了什么?

任务四【主交付物】：交互式密文伪造工具

目的: 把攻击做成一个可复用的工具。**这是本实验的主要交付物。**

```
python task4_forge.py
```

程序会:

1. 列出历史密文库里的所有密文
2. 询问你要伪造哪一条消息
3. 询问你要改哪个字段 (From / To / Date / Place)
4. 列出该字段上可选的来源消息 (自动排除“换了等于没换”的)
5. 生成伪造密文并写入 data/forged.ct

然后由 Bob 来解密:

```
python bob_decrypt.py -i data/forged.ct
```

实测效果

对 M1 改 Place，从 M2 取块:

```
伪造密文:
08096a96a3a0475152d184e5931b4b1a43f16df7956aa0ba8a53ece25932f60b
90cd57ec0e9013c73633feca93e40681afed40e112212decde60a35b5fc8422e
```

Bob 解密:

```
=====
Bob 收到密文: data/forged.ct
长度 64 字节 = 4 个分组
=====
```

```
解密结果:
-----
From: Alice
To: Bob
Date: 2026-03-15
Place: Gym
-----
```

```
字段解析:
sender  = Alice
to      = Bob
date    = 2026-03-15
place   = Gym
```

⚠ Bob 无法判断这条密文是不是原装的 — ECB 没有任何完整性保护。

要求

1. 跑通并截图攻击链路: task4_forge.py → bob_decrypt.py。
2. 验证无密钥: 运行 3.3 节的 AST 检查命令, 证明 task4_forge.py 没有引入 ecb_common。
3. 扩展功能 (至少实现一项):
 - 支持一次改多个字段
 - 支持从 3 条以上消息拼装
 - 自动搜索: 给定目标明文, 自动找出能拼出它的分组组合
 - 加入“删除一个分组”的攻击 (想想解密会发生什么)
4. 回答: 为什么 bob_decrypt.py 无法察觉密文被改过? 如果要让它能察觉, 需要额外传什么信息?

任务五【挑战】: 把攻击做到极致

```
python task5_challenge.py
```

挑战 1: 一次改多个字段, 消息仍然通顺

把 M1 (Alice -> Bob, 03-15, Library) 改成「Carol 在 03-22 通知 Bob 去 Gym」, 一次改三个字段。

实测取块配方:

分组	取自	内容
分组0	M4	From: Carol
分组1	M1	To: Bob
分组2	M3	Date: 2026-03-22
分组3	M2	Place: Gym

伪造密文:

```
8a75dc86c8b472ad42785f678d721fdb43f16df7956aa0ba8a53ece25932f60b
b7ad3eb18b72990bd67d73adcb0ea5f4afed40e112212decde60a35b5fc8422e
```

解密结果:

```
From: Carol
To: Bob
Date: 2026-03-22
Place: Gym
```

挑战 2: 用至少 3 条消息的密文块拼出一条全新消息

要求分组来自 ≥ 3 条不同消息。实测用 4 条:

分组	取自	内容
分组0	M5	From: Bob
分组1	M6	To: Carol
分组2	M3	Date: 2026-03-22
分组3	M2	Place: Gym

结果:

```
From: Bob
To: Carol
Date: 2026-03-22
Place: Gym
```

这条消息**从来没有任何人发送过**——它是四个分组的拼装件，但格式完整、语义自治、解密不报错。

要求

- 完成两个挑战，提交你的取块配方与实测输出。
- 思考**: 如果消息里加一个「消息序号」字段（每次递增），这个攻击会受什么影响？攻击者有没有办法绕过？
- 思考**: 如果消息里加一个「时间戳」，Mallory 能做些什么？（提示：他能不能把一条**旧**消息重新发出去？）

任务六：防御对照

目的：搞清楚“换成 CBC 就安全了吗”这个问题的真正答案。

```
python task6_defense.py
```

程序用同一条“搬分组”的动作，在三种模式下各跑一遍。

对照 1: ECB

搬分组 → **成功**，Library 变 Gym。

对照 2: CBC，两条消息前缀不同 (M1 vs M4)

搬分组 → **失败**:

```
分组0: |From: Alice      | <- 正常
分组1: |To: Bob          | <- 正常
分组2: |Date: 2026-03-15| <- 正常
分组3: |*~ 3.....I...o| <- 乱码（前一个密文分组对不上）
```

CBC: 被拒绝 -> ValueError: 填充字节非法: 0x6f (PKCS#7)

原因是 CBC 有链接: $ct[i] = E(pt[i] \oplus ct[i-1])$ ，每个分组都绑定前驱。搬来的分组对不上前驱，解出乱码。

对照 3【重点】：CBC，但两条消息前缀相同 (M1 vs M2)

搬分组 → **成功**:

```
M1 与 M2 前三个分组相同 -> 密文前三个分组也相同:
ct1[2] = e1e8f042a2aa8819853f24c3bd5c0832
ct2[2] = e1e8f042a2aa8819853f24c3bd5c0832
相等: True
```

```
CBC: 解密成功
From: Alice
To: Bob
Date: 2026-03-15
Place: Gym
```

这个结果非常重要:

CBC 并不阻止分组搬运。它只让「前驱对不上」的情况失败，而前缀相同的两条消息，前驱天然对得上。

只要攻击者能找到两条前缀相同的消息，同样的攻击在 CBC 下照样成立。

对照 4: GCM —— 分组照样能搬，但标签会揭穿

密文部分 63 字节 (= 明文长度, 无填充)
认证标签 16 字节: e39e74cd84ab434d377c9d4f02424019

Mallory 拿搬完的密文, 配上他推算出的密钥流:
分组3 明文 = |Place: Gym | <- 分组层面**搬成功了**

GCM: 被拒绝 -> InvalidTag: 认证标签校验失败

看清楚: GCM 在**分组层面同样挡不住搬运** (它的密文部分就是流密码, 每个字节只和自己位置上的密钥流有关)。拦住攻击的是那 16 字节认证标签 —— 它覆盖了整条密文, 任何一比特被改动都会导致校验失败。

要求

1. 三种模式的实测结果都要在报告里体现。
2. **回答:** 为什么说“CBC 的错误扩散”不是完整性保护? (提示: 想想填充校验被随机字节蒙混过关的概率是多少)
3. **回答:** GCM 的认证标签为什么必须**在解密之前**校验? 如果先解密再看内容, 会有什么问题?

5. 实验报告要求

报告需包含以下部分, **请务必附上实际运行的截图或输出文本**:

5.1 必做内容

1. **环境自检结果** (check_env.py 的输出)
2. **任务一 ~ 任务三:** 程序输出 + 你对各思考题的回答
3. **任务四:**
 - 伪造工具的运行截图 (完整攻击链路)
 - AST 检查的输出 (证明无密钥)
 - 你的扩展功能说明与代码
4. **任务五:** 两个挑战的取块配方与实测结果
5. **任务六:** 三种模式的对照结果与分析

5.2 分析题 (重点)

6. 用自己的话解释: **为什么 AES 本身是安全的, 但用 ECB 加密却能被伪造?** 请明确区分“分组密码”与“工作模式”的责任。
7. 本实验用的是“一个字段一个分组”的简单格式。如果格式更复杂 (比如字段跨多个分组、有嵌套结构), 攻击的难度会怎样变化? 攻击者需要额外知道什么?

5.3 报告格式

- 提交 PDF
- 关键代码片段直接贴进报告, 不要只给文件名
- 所有结论都要有**可复核的证据** (输出、截图、十六进制数据)

6. 思考题

1. **格式与安全的关系** 格式 B (定长对齐) 在工程上更优, 在安全上更差。这个矛盾在真实系统里常见吗? 举一个你熟悉的例子。
2. **“加密”这个词的歧义** 很多人说“这个文件我加密了”就认为安全了。本实验说明了什么? “加密”最少要包含哪几件事才算完整?
3. **确定性加密的正当用途** ECB 是确定性的, 因此会泄露模式。但有些场景需要确定性加密 (比如加密数据库索引, 要能等值查询)。请查资料了解“确定性加密”以及“可搜索加密”, 说说它们如何在使用确定性带来的便利的同时控制风险。

附录 A: 环境与排错

A.1 依赖安装问题

现象	原因	解决
ModuleNotFoundError: No module named 'cryptography'	没装	python -m pip install cryptography
装完还是找不到	装到了别的解释器	用 python -m pip install, 或检查 which python

公司/学校网络装不上	网络代理	<code>pip install -i https://pypi.tuna.tsinghua.edu.cn/simple cryptography</code>
------------	------	---

A.2 运行问题

现象	原因	解决
找不到 <code>data/history.json</code>	没建历史库	<code>python alice_encrypt.py --build-history</code>
<code>ValueError</code> : 数据长度不是分组长度的整数倍	忘了 PKCS#7 填充	ECB/CBC 不会 自动填充, 要用 <code>padding.PKCS7(128)</code>
<code>ValueError</code> : 填充字节非法	密文被改动过	这正是任务六要观察的现象, 不是 bug
明文里出现 <code>\x01</code> 等奇怪字符	你看到的是填充字节	用 <code>pretty_aligned()</code> 或先 <code>unpad</code>
<code>ValueError</code> : 日期必须恰好 10 位	格式 B 的约束	日期必须是 YYYY-MM-DD
<code>ValueError</code> : 地点过长	格式 B 的约束	地点最多 8 字符 (这是让字段正好占一个分组的前提)

A.3 关于“文档里的数字和我跑的不一样”

AES-ECB 是确定性的, 同一个密钥、同一段明文, 在任何实现上都得到 **逐字节相同**的密文。所以:

- 如果你跑出的密文与 附录 B 不同, 请先检查**密钥派生**和**消息格式**是否与 `ecb_common.py` 一致;
- 如果你改了密钥或格式, 那数字当然会变——这是正常的, 重要的是**攻击是否依然成立**。

本实验包中, `tools/gen_truth.py` 生成附录 B 的全部数据, 而 `tools/aes_ref.py` 通过 FIPS-197 与 NIST SP 800-38A 的官方向量自校验。也就是说, 附录 B 里的每个数字都可以在**不联网、不装任何第三方包**的机器上离线复核。

A.4 Windows 下中文显示异常

这一条**不影响任何计算结果**, 纯粹是显示问题。

现象	原因	解决
直接运行时中文正常, 但重定向到文件后 (python <code>task1_basics.py > out.txt</code>) 用 VS Code 打开是乱码	中文 Windows 的 <code>cmd.exe</code> 输出编码是 cp936 (GBK) , 而 VS Code 默认按 UTF-8 打开	先 <code>chcp 65001</code> 切到 UTF-8 再运行; 或设环境变量 <code>PYTHONIOENCODING=utf-8</code>
输出里有 ? 代替了  之类的符号	该符号不在 GBK 字符集里, 被替换掉了	正常现象, 不影响结果; 想看到就 <code>chcp 65001</code>
<code>UnicodeEncodeError: 'gbk' codec can't encode character</code>	打印了 GBK 装不下的字符	用上面的 <code>chcp 65001</code> 或 <code>PYTHONIOENCODING=utf-8</code>

`run_all.py` 已经在内部做了两件事避免这类麻烦: 它强制子进程用 UTF-8 输出, 并在打印时把无法编码的字符降级替换, 因此它**不会**因为编码问题崩溃。

⚠ 如果你要做思考题里“用中文消息重做实验”这一条: 中文字符在 UTF-8 下 每个占 3 字节, **不再与 16 字节的分组边界对齐**, 格式 B 的前提会被打破。这正是那条思考题想让你发现的——先 `chcp 65001` 再跑, 否则你会先撞上乱码, 误以为是程序的问题。

附录 B: 真值数据 (实测)

本附录的数据由 `tools/gen_truth.py` 生成 (纯标准库 AES 实现), 与使用 `cryptography` 库的运行结果**逐字节相同**。复核命令:
`cd tools && python gen_truth.py && python aes_ref.py`

B.1 演示密钥

```
DEMO_KEY = SHA-256("cryptography-course/ecb-demo/v1")[:16]
           = 272af8e3a0340079ef68280aa145165e
```

真实系统里密钥来自密钥管理系统。这里用一个固定派生值, 是为了让**全班跑出的密文一致**, 便于批改和讨论。

B.2 格式 A 的分组布局 (M1 与 M3 对照)

```
f"From: {sender}\nTo: {to}\nDate: {date}\nPlace: {place}"
```

消息	明文长度	填充后	分组数
----	------	-----	-----

M1	51	64	4
M2	47	48	3
M3	51	64	4
M4	51	64	4
M5	48	64	4
M6	53	64	4

M1 与 M3 逐分组对照（二者只差日期）：

分组	M1 明文	M3 明文	密文相同?
0	From: Alice·To:	From: Alice·To:	✅ 相同
1	Bob·Date: 2026-0	Bob·Date: 2026-0	✅ 相同
2	3-15·Place: Libr	3-22·Place: Libr	❌ 不同
3	ary + 13×0x0D	ary + 13×0x0D	✅ 相同

注意分组 3：两边都是 ary 加填充，所以换了等于没换。而分组 2 里夹着日期的尾巴和地点的开头——换它必然连带改动两个字段。

错位示例（M1 与 M2 长度不同）：

把 M2 的最后一个分组接到 M1 的第 3 分组之后，解密得到：

From: Alice
To: Bob
Date: 2026-03-15
Place: Libr3-15
Place: Gym

B.3 格式 B：历史密文库

data/history.json 中的 6 条消息：

编号	发件人	收件人	日期	地点
M1	Alice	Bob	2026-03-15	Library
M2	Alice	Bob	2026-03-15	Gym
M3	Alice	Bob	2026-03-22	Library
M4	Carol	Dave	2026-03-15	Office
M5	Bob	Alice	2026-04-01	Cafe
M6	Alice	Carol	2026-03-15	Library

明文分组（· 表示填充字节 0x01）：

消息	分组0	分组1	分组2	分组3
M1	From: Alice	To: Bob	Date: 2026-03-15	Place: Library ·
M2	From: Alice	To: Bob	Date: 2026-03-15	Place: Gym ·
M3	From: Alice	To: Bob	Date: 2026-03-22	Place: Library ·
M4	From: Carol	To: Dave	Date: 2026-03-15	Place: Office ·
M5	From: Bob	To: Alice	Date: 2026-04-01	Place: Cafe ·
M6	From: Alice	To: Carol	Date: 2026-03-15	Place: Library ·

密文分组（十六进制）：

消息	分组0	分组1	分组2	分组3
M1	08096a96a3a0475152d184e5931b4b1a	43f16df7956aa0ba8a53ece25932f60b	90cd57ec0e9013c73633feca93e40681	050d523d6!
M2	08096a96a3a0475152d184e5931b4b1a	43f16df7956aa0ba8a53ece25932f60b	90cd57ec0e9013c73633feca93e40681	afed40e11!
M3	08096a96a3a0475152d184e5931b4b1a	43f16df7956aa0ba8a53ece25932f60b	b7ad3eb18b72990bd67d73adcb0ea5f4	050d523d6!

M4	8a75dc86c8b472ad42785f678d721fdb	9bc0063c59de33b9212d2bdd3ca9d3f9	90cd57ec0e9013c73633feca93e40681	80f2e79act
M5	f72917986023578132aba98029dd28e7	e4c0e1f232a008f167512b19795f50ef	0fc67efdf61a98fd0b7b5dbc4c57ab67	3c7df188c4
M6	08096a96a3a0475152d184e5931b4b1a	ac1f9681fc3a7c6817285ba00b1b8463	90cd57ec0e9013c73633feca93e40681	050d523d6f

B.4 攻击配方一览

编号	取块配方	解密结果
攻击 1	M1.0 + M1.1 + M1.2 + M2.3	Alice → Bob, 2026-03-15, Gym
攻击 2	M4.0 + M1.1 + M1.2 + M1.3	Carol → Bob, 2026-03-15, Library
攻击 3	M1.0 + M1.1 + M3.2 + M1.3	Alice → Bob, 2026-03-22, Library
挑战 1	M4.0 + M1.1 + M3.2 + M2.3	Carol → Bob, 2026-03-22, Gym
挑战 2	M5.0 + M6.1 + M3.2 + M2.3	Bob → Carol, 2026-03-22, Gym

B.5 CBC 防御对照实测

ECB 与 CBC 的唯一区别是链接：

ECB:	ct[i] = E(pt[i])	分组之间互不相干
CBC:	ct[i] = E(pt[i] XOR ct[i-1])	分组与前一个密文分组绑定

对照	场景	结果
1	M1 与 M4（前缀就不同），搬 最后 一个分组 3	❌ 被拒：填充字节非法：0x6f
2	同上，搬 中间 的分组 1	⚠️ 填充校验碰巧通过 ，但分组 1 及其后全部是乱码（ 错误扩散 ）
3	M1 与 M2（ 前缀相同 ），搬分组 3	⚠️ 完全成功 ：Library 变 Gym
4	GCM，搬分组 3	❌ 被拒：InvalidTag（标签校验不过）

⚠️ **对照 2 值得特别注意**。改动中间分组时，填充校验**通过了**——但那只是运气：随机字节有约 1/256 的概率恰好构成合法填充。Bob 拿到的是一条肉眼可见的乱码消息。

这说明：**填充校验从来就不是完整性校验**。它只是碰巧能挡住一部分篡改，挡不住的部分就会被放行。

对照 3 是本节的重点：CBC 并不阻止分组搬运，它只让“前驱对不上”的情况失败。前缀相同的两条消息，前驱天然对得上。

对照 4 同样是重点：GCM 在**分组层面**也挡不住搬运（它的密文部分就是流密码，逐字节独立），真正拦住攻击的是认证标签。

B.6 防御方案对照

方案	能否挡住本实验的攻击	说明
ECB	❌ 完全挡不住	分组独立 + 无完整性
ECB + HMAC	✅ 能	但 ECB 本身仍泄露模式，不应使用
CBC（随机 IV）	⚠️ 大幅变难，但 不是保证	依赖填充校验兜底，约 1/256 概率被蒙混
CBC + HMAC	✅ 能	Encrypt-then-MAC，先验 MAC 再解密
AES-GCM	✅ 能	AEAD，认证标签覆盖整条密文
ChaCha20-Poly1305	✅ 能	同上

正确结论：

- IV / nonce 必须每次随机且不可预测、不可重用；
- 加密**必须**配合完整性保护；
- 完整性必须在**解密之前**验证（先认证，后解密）；
- 不要用 ECB，任何场景都不要——它连“看起来安全”都做不到。

附录 C：知识点映射

课程知识点	在本实验中的体现	位置
分组密码与置换	AES 是固定密钥下的双射；确定性由密钥固定而来	1.1

工作模式	ECB / CBC / CTR / GCM 的对比	1.2、任务六
PKCS#7 填充	为什么 51 字节变 64 字节；填充不是完整性保护	任务一、任务六
异或运算	CBC 的链接、CTR/GCM 的密钥流	任务六
认证加密（AEAD）	GCM 的认证标签；"先认证后解密"	任务六
已知明文攻击	Mallory 对历史消息内容的了解	1.6
密码学协议设计	消息格式本身就是安全设计的一部分	任务三、思考题 1
随机性与 IV	固定 IV / nonce 重用的危害	任务六
密钥管理	攻击者程序的依赖隔离（无密钥也能伪造）	3.3、任务四

附录 D：一页速查

如果你只记得住一件事，请记住这个：

加密不等于安全。

「机密性」（别人看不到）和「完整性」（别人改不了）是**两件事**，必须**分别**获得。AES-ECB 提供了机密性，一点完整性都没有。

如果你记得住三件事：

1. **ECB 是确定性的** → 相同明文块产生相同密文块 → 泄露消息结构。
2. **ECB 无完整性** → 密文块可以像积木一样搬运 → 不需要密钥就能伪造。
3. **修法是认证加密** → AES-GCM / ChaCha20-Poly1305，或者 Encrypt-then-MAC，并且**先认证后解密**。

本指导书所有实验数据均为实测，可通过 `tools/` 下的脚本离线复核。